

目录

业务逻辑与越权漏洞修复报告	1
目录	2
1. 漏洞概述	2
2. 漏洞一：IDOR 越权访问	2
2.1 问题描述	2
2.2 问题代码	2
2.3 攻击示例	3
2.4 危害等级	3
3. 漏洞二：充值金额无校验	3
3.1 问题描述	3
3.2 问题代码	3
3.3 攻击示例	3
3.4 危害等级	4
4. 漏洞三：越权充值	4
4.1 问题描述	4
4.2 攻击示例	4
5. 漏洞四：密码信息泄露	4
5.1 问题描述	4
6. IDOR 漏洞原理	5
6.1 什么是 IDOR	5
6.2 攻击流程	5
6.3 为什么发生	5
7. 修复方案	5
7.1 修复对照表	5
7.2 修复后核心代码	6
8. 修复前后代码对比	7
个人中心—越权修复	7
充值—金额校验 + 越权修复	8
9. 安全建议	9
9.1 权限校验原则	9
9.2 开发者检查清单	9
9.3 业务逻辑安全规则	9
9.4 OWASP 参考	9

业务逻辑与越权漏洞修复报告

项目名称：用户信息管理系统
报告日期：2026-07-09
审计目标：Flask 用户管理平台中的业务逻辑漏洞与越权漏洞分析与修复
报告人：安全审计组

目录

1. 漏洞概述
2. 漏洞一：IDOR 越权访问
3. 漏洞二：充值金额无校验
4. 漏洞三：越权充值
5. 漏洞四：密码信息泄露
6. IDOR 漏洞原理
7. 修复方案
8. 修复前后代码对比
9. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的个人中心和充值功能存在 **4 项安全漏洞**，全部与业务逻辑和权限控制相关。

编号	漏洞名称	风险等级	漏洞类型
BL-001	IDOR 越权查看资料	高危	越权漏洞
BL-002	充值金额无正负校验	高危	业务逻辑漏洞
BL-003	越权为他人充值	高危	越权 + 业务逻辑
BL-004	个人中心泄露密码明文	中危	信息泄露

2. 漏洞一：IDOR 越权访问

2.1 问题描述

IDOR (Insecure Direct Object Reference, 不安全的直接对象引用) 指应用程序直接使用用户提供的参数 (如 ID) 来访问对象, 但没有验证当前用户是否有权限访问该对象。

2.2 问题代码

```
@app.route("/profile", methods=["GET"])
def profile():
    user_id = request.args.get("user_id", type=int)
    user_data = get_user_by_id(user_id) if user_id else None
    return render_template("profile.html", user=user_data)
```

2.3 攻击示例

不需要登录，直接访问：

URL	结果
/profile?user_id=1	查看 admin 的资料（含密码）
/profile?user_id=2	查看 alice 的资料（含密码）
/profile?user_id=3	遍历所有注册用户资料

攻击者只需在浏览器地址栏修改 `user_id` 参数的值，即可查看系统中任意用户的个人信息，包括密码明文。

2.4 危害等级

攻击类型	可能性	影响
用户信息泄露	极高	获取任意用户的手机、邮箱、密码
批量信息收集	高	遍历 ID 可获取全部用户信息

3. 漏洞二：充值金额无校验

3.1 问题描述

充值接口没有检查 `amount` 参数的正负，用户可以充入任意数值，包括负数。

3.2 问题代码

```
@app.route("/recharge", methods=["POST"])
def recharge():
    user_id = request.form.get("user_id", type=int)
    amount = request.form.get("amount", type=float)

    # 直接修改余额，不检查 amount 是否合法
    data["balance"] += amount    # amount 可以是负数!
```

3.3 攻击示例

操作	金额	余额变化	结果
正常充值	+100	100 → 200	☒ 正常

操作	金额	余额变化	结果
恶意扣款	-10000	100 → -9900	☒ 余额为负
刷分	+999999	100 → 1000099	☒ 无限刷钱

攻击者通过修改 POST 请求中的 amount 参数：

```
POST /recharge
user_id=1&amount=-99999    ← 扣光管理员余额
```

```
POST /recharge
user_id=1&amount=9999999    ← 给自己无限加钱
```

3.4 危害等级

攻击类型	可能性	影响
恶意扣款	极高	任何用户余额可被清零
刷钱漏洞	极高	无限增加余额
业务损失	高	严重的经济损失

4. 漏洞三：越权充值

4.1 问题描述

充值接口只接收 user_id 参数，但没有验证当前登录用户是否与要充值的用户一致。

4.2 攻击示例

```
POST /recharge
user_id=1&amount=-99999    ← 给 admin 扣款（任意用户都能操作）
```

攻击者可构造请求给任意用户充值或扣款，即使不是自己的账号。

5. 漏洞四：密码信息泄露

5.1 问题描述

个人中心页面直接显示用户的密码明文。

```
<!-- profile.html -->
<li>
  <span class="user-info-label">密码</span>
  <span class="user-info-value">{{ user['password'] }}</span>
</li>
```

结合 IDOR 漏洞，攻击者可以批量获取所有用户的密码。

6. IDOR 漏洞原理

6.1 什么是 IDOR

IDOR (Insecure Direct Object Reference) 属于 OWASP Top 10 中 A01:2021 (失效的访问控制) 的一种。它发生在应用程序直接使用用户控制的输入来访问对象，但没有验证用户权限时。

6.2 攻击流程

用户 → 访问 /profile?user_id=2
↓
服务器 → SELECT * FROM users WHERE id = 2
↓
→ 返回 alice 的完整资料
↓
→ 用户看到 alice 的密码和余额

6.3 为什么发生

原因	说明
信任客户端	认为用户不会修改 URL 参数
缺乏权限校验	没有检查“当前用户”和“被访问资源”的关系
参数来源不统一	user_id 从 URL 获取而不是从 session 获取

7. 修复方案

7.1 修复对照表

漏洞编号	修复措施	风险等级变化
BL-001	验证 session 中的 user_id 与请求的 user_id 一致	高危 → 已修复
BL-002	amount 必须大于 0	高危 → 已修复
BL-003	充值接口验证当前登录用户的身份	高危 → 已修复
BL-004	删除个人中心的密码显示	中危 → 已修复

7.2 修复后核心代码

```
# ===== 个人中心（权限校验） =====
@app.route("/profile", methods=["GET"])
def profile():
    if "username" not in session:
        return redirect("/login")

    user_id = request.args.get("user_id", type=int)

    # 【已修复】IDOR 防护：检查当前登录用户是否与请求的 user_id 匹配
    if session.get("user_id") != user_id:
        return render_template("profile.html", user=None, error=" 无权查看其他用户的资料")

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))
    row = cursor.fetchone()
    conn.close()

    if row:
        # 【已修复】不返回 password 字段
        user_data = {
            "id": row["id"],
            "username": row["username"],
            "email": row["email"],
            "phone": row["phone"],
            "role": "user",
            "balance": row["balance"] if row["balance"] else 0,
        }
        return render_template("profile.html", user=user_data)
    return render_template("profile.html", user=None, error=" 用户不存在")
```

```

# ===== 充值（金额校验 + 权限校验） =====
@app.route("/recharge", methods=["POST"])
def recharge():
    if "username" not in session:
        return redirect("/login")

    user_id = request.form.get("user_id", type=int)
    amount = request.form.get("amount", type=float)

    # 【已修复】IDOR 防护：只能给自己的账户充值
    if session.get("user_id") != user_id:
        return render_template("profile.html", user=None, error=" 无权给其他用户充值")

    # 【已修复】金额校验：amount 必须为正数
    if not amount or amount <= 0:
        return redirect(f"/profile?user_id={user_id}")

    conn = sqlite3.connect("data/users.db")
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE users SET balance = COALESCE(balance, 0) + ? WHERE id = ?",
        (amount, user_id),
    )
    conn.commit()
    conn.close()

    return redirect(f"/profile?user_id={user_id}")

```

8. 修复前后代码对比

个人中心—越权修复

```

# ===== 漏洞版（无权限校验） =====
@app.route("/profile")
def profile():
    user_id = request.args.get("user_id", type=int)
    user_data = get_user_by_id(user_id) # 直接返回任何用户的数据
    return render_template("profile.html", user=user_data)

# ===== 修复版（session 校验） =====
@app.route("/profile")

```

```
def profile():
    if "username" not in session:
        return redirect("/login")
    user_id = request.args.get("user_id", type=int)
    if session.get("user_id") != user_id:
        return render_template("profile.html", user=None, error=" 无权查看")
    # 只有自己才能查看自己的资料
```

充值—金额校验 + 越权修复

```
# ===== 漏洞版（无校验） =====
@app.route("/recharge", methods=["POST"])
def recharge():
    user_id = request.form.get("user_id", type=int)
    amount = request.form.get("amount", type=float)
    data["balance"] += amount # 任意金额，任意用户
    return redirect(f"/profile?user_id={user_id}")

# ===== 修复版（双重校验） =====
@app.route("/recharge", methods=["POST"])
def recharge():
    if "username" not in session:
        return redirect("/login")
    user_id = request.form.get("user_id", type=int)
    amount = request.form.get("amount", type=float)

    # 校验：只能给自己充值
    if session.get("user_id") != user_id:
        return render_template("profile.html", user=None, error=" 无权操作")

    # 校验：金额必须为正数
    if not amount or amount <= 0:
        return redirect(f"/profile?user_id={user_id}")

    # 参数化查询更新余额
    conn = sqlite3.connect("data/users.db")
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE users SET balance = COALESCE(balance, 0) + ? WHERE id = ?",
        (amount, user_id),
    )
    conn.commit()
    conn.close()
```

```
return redirect(f"/profile?user_id={user_id}")
```

9. 安全建议

9.1 权限校验原则

用户请求 → 识别身份 (Session/Token)

↓

获取参数 (URL/表单)

↓

验证权限：当前用户是否有权操作此资源？

是 → 执行操作

否 → 返回 403 禁止访问

9.2 开发者检查清单

- ☐ 所有通过参数指定 ID 的接口都必须做权限校验
- ☐ user_id 优先从 session 获取，而不是从 URL 参数获取
- ☐ 任何金额操作必须校验正负和合法性
- ☐ 敏感信息（密码、手机号）不应在页面上展示
- ☐ 不要信任客户端提交的任何数据
- ☐ 使用 session 中的身份信息做权限判断

9.3 业务逻辑安全规则

规则	说明	示例
身份来源固定	用户身份从 session/token 获取，不从请求参数获取	user_id 从 session['user_id'] 获取
资源归属校验	每次访问资源都要校验所有权	只能查看自己的订单/资料
金额操作校验	涉及金额的操作用正数校验 + 上限控制	amount > 0 AND amount <= 10000
敏感信息过滤	API 和页面不返回不需要的敏感字段	不返回 password、id_card 等

9.4 OWASP 参考

IDOR 属于 OWASP Top 10 A01:2021 - 失效的访问控制 (Broken Access Control)，是 Web 应用中最常见的安全漏洞之一。

修复 IDOR 的核心原则：**永远不要信任客户端提供的对象引用，始终验证用户对请求资源的访问权限。**

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A01:2021 Broken Access Control

测试环境： Flask 用户管理系统

免责声明： 本报告仅用于安全学习和教学目的